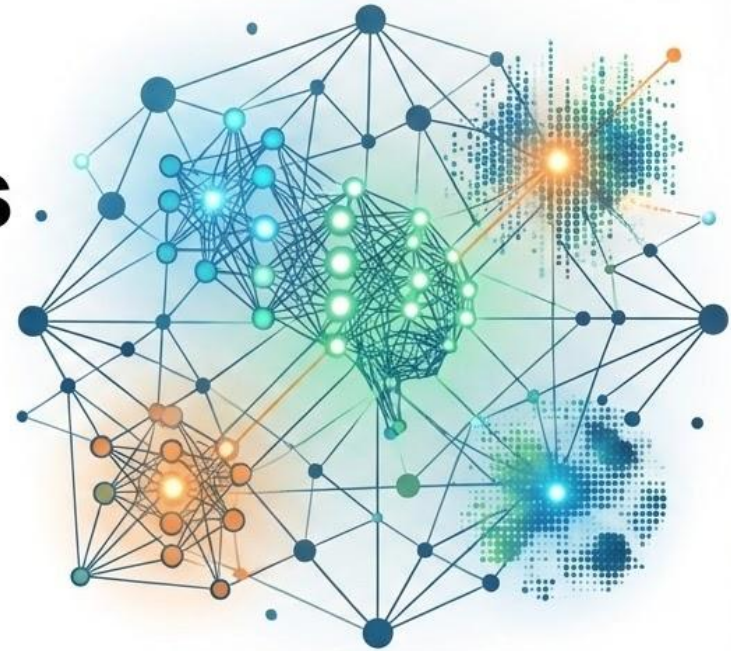


Diplomatura en python para la ciencia de datos y machine learning

Módulo 1 – Desarrollo
Avanzado en Python

2- Estructuras



Listas

Las listas son secuencias ordenadas de objetos. Estos objetos pueden ser datos de cualquier tipo: strings, integers, floats, booleanos, listas, entre otros. Son tipos de datos mutables.



Mutable



Ordenado



Admite Duplicados

Ejemplos de Listas

```
lista_1 = ["C", "C++", "Python", "Java"]  
lista_2 = ["PHP", "SQL", "Visual Basic"]
```

Indexado: podemos acceder a los elementos de una lista a través de sus índices [inicio:fin:paso]

```
print(lista_1[1:3])  
>> ["C++", "Python"]
```

Cantidad de elementos: a través de la propiedad len()

```
print(len(lista_1))  
>> 4
```

Concatenación: sumamos los elementos de varias listas con el símbolo +

```
print(lista_1 + lista_2)  
>> ['C', 'C++', 'Python', 'Java', 'PHP', 'SQL', 'Visual Basic']
```

La función **range()** devuelve una secuencia de números dados 3 parámetros. Se utiliza fundamentalmente para controlar el número de ejecuciones de un loop o para crear rápidamente una serie de valores.

número a partir del cual
inicia el rango (**incluido**)

diferencia entre cada valor
consecutivo de la secuencia

`range(valor_inicio, valor_final, paso)`

número antes del cual el
rango finaliza (**no incluido**)

```
print(list(range(1,13,3)))  
>> [1,4,7,10]
```

Nota: un parámetro obligatorio es **valor_final**. Los valores predeterminados para **valor_inicio** y **paso** son 0 y 1 respectivamente.

Operaciones que no modifican una lista

Medición y Búsqueda

 `len(l)` : Devuelve el número de elementos de la lista `l`.

 `l.index(dato)` : Devuelve la posición del primer elemento con valor `dato`.

 `dato in l` : Devuelve `True` si `dato` pertenece a `l`, `False` si no.

```
>>> a = [1, 2, 2, 3]
>>> len(a) → 4
>>> a.index(2) → 1
>>> 3 in a → True
```

Valores Extremos y Suma

 `min(l)` : Devuelve el mínimo elemento de la lista `l`.

 `max(l)` : Devuelve el máximo elemento de la lista `l`.

Σ `sum(l)` : Devuelve la suma de los elementos de la lista `l`.

```
>>> min(a) → 1
>>> max(a) → 3
>>> sum(a) → 8
```

Lógica y Conteo ✓

 `l.count(dato)` : Devuelve el número de veces que `dato` aparece en `l`.

 `all(l)` : Devuelve `True` si todos los elementos de `l` son `True`, `False` si no.

```
>>> a.count(2) → 2
>>> all([True, 1, 'yes']) → True
>>> any([0, False, 3<2]) → False
```


Otras operaciones

Concatenación de Listas

```
>>> l1 = [1, 3, 5]
>>> l2 = [2, 4, 6]
>>> l1 + l2
[1, 3, 5, 2, 4, 6]
```

Modificación y Adición

```
>>> a = [1, 3, 5]
>>> a.append(5) # Añade dato al final
>>> a
[1, 3, 5, 5]
>>> a.extend([2, 4, 6]) # Añade elementos de secuencia a
>>> a
[1, 3, 5, 5, 2, 4, 6]
>>> a.insert(1, 3) # Inserta dato en posición índice
>>> a
[1, 3, 3, 5, 5, 2, 4, 6]
```

Eliminación

```
>>> a.remove(3) # Elimina la primera aparición de dato
>>> a
[1, 3, 5, 5, 2, 4, 6]
>>> a.pop() # Devuelve y elimina el último elemento
6
>>> a
[1, 3, 5, 5, 2, 4]
```

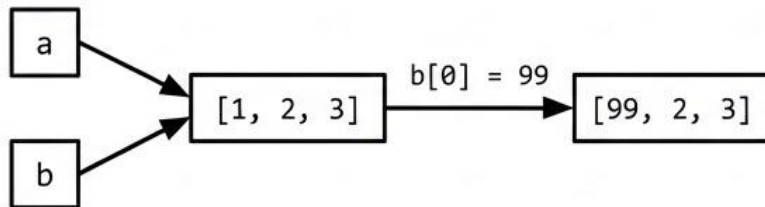
Ordenamiento

```
>>> a.sort() # Ordena la lista
>>> a
[1, 2, 3, 4, 5, 5]
>>> a.reverse() # Invierte el orden de la lista
>>> a
[5, 5, 4, 3, 2, 1]
```

Copiar listas

Copia por Referencia (`l1 = l2`)

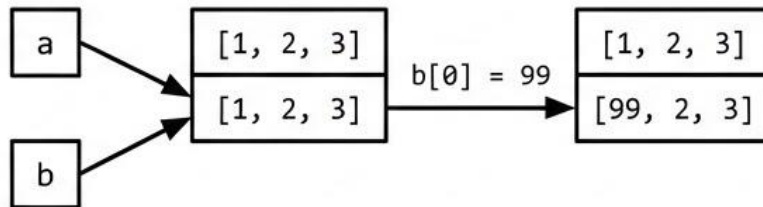
Asocia la nueva variable a la *misma* dirección de memoria. Los cambios afectan a ambas variables.



```
>>> a = [1, 2, 3]
>>> b = a          # Referencia
>>> b[0] = 99
>>> print(a)
[99, 2, 3]          # 'a' también cambia
```

Copia por Valor (`l1 = l2.copy()`)

Crea una *nueva* lista en una dirección de memoria diferente. Los cambios son independientes.



```
>>> a = [1, 2, 3]
>>> b = a.copy()   # Valor
>>> b[0] = 99
>>> print(a)
[1, 2, 3]           # 'a' NO cambia
```

Tuplas

Características Principales



Inmutables: No pueden modificarse tras su creación.



Ordenadas: Los elementos mantienen su orden de inserción.



Admiten Duplicados: Pueden contener elementos repetidos.

Creación y Acceso

```
mi_tuple = (1, 'dos', 3.33, 'cuatro', (5.0, 6))  
print(mi_tuple[3]) # Salida: cuatro
```

Conversivos

```
t = (1, 2, 3, 1, 1)  
print(t.count(1)) # Salida: 3  
print(t.index(2)) # Salida: 1
```

Inmutabilidad

```
# Intentar modificar genera error  
# mi_tuple[0] = 'UNO'  
# TypeError: 'tuple' object does not  
# support item assignment
```

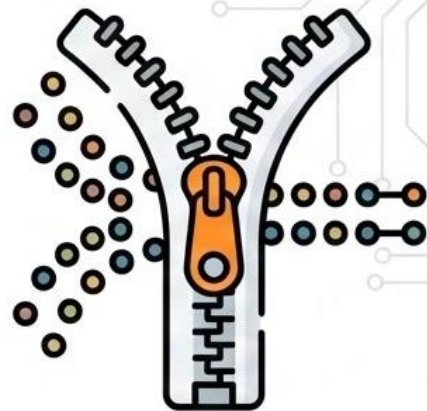
Métodos Básicos

```
t = (1, 2, 3, 1, 1)  
print(t.count(1)) # Salida: 3  
print(t.index(2)) # Salida: 1
```

zip

Concepto y Analogía

- La función `zip()` crea un iterador de tuplas, emparejando elementos de varios iterables por su índice. Su nombre proviene de *zipper* (cremallera), como una analogía visual.
- Se detiene cuando el iterable más corto se agota.



```
letras = ['w', 'x', 'c']  
numeros = [50, 65, 90, 110, 135]  
for letra, num in zip(letras, numeros):  
    print(f'Letra: {letra}, y Número: {num}')
```

```
Letra: w, y Número: 50  
Letra: x, y Número: 65  
Letra: c, y Número: 90
```


Diccionario

Conceptos Básicos y Creación

Estructuras de datos que almacenan información en pares **clave:valor**. Útiles para guardar y recuperar datos por nombre de clave, no por índice.

Mutable 

Ordenado 
(Python 3.7+)

Claves Únicas  / Valores Duplicados 

valor
clave

```
mi_diccionario = {"curso": "Python TOTAL", "clase": "Diccionarios"}
```

Ejemplo de Uso

```
mi_diccionario["recursos"] = ["notas", "ejercicios"]
```

Ejemplo de Uso

```
print(mi_diccionario["recursos"][1]) # Salida: ejercicios  
>> "ejercicios"
```

mi_diccionario → keys() values() items()

* A partir de Python 3.7+, los diccionarios son tipos de datos ordenados, en el sentido que dicho orden se mantiene según su orden de inserción para aumentar la eficiencia en el uso de la memoria.

Funciones y Operaciones

Las funciones **min()** y **max()** retornan el ítem con el valor más bajo o más alto, respectivamente. En strings, la comparación es alfabética.

```
ciudades_habitantes = {"Tijuana": 1810645, "León": 1579883}
```

```
lista_valores = [5**5, 12**2, 3050, 475*2]
```

Operaciones con min() y max()

```
print(min(ciudades_habitantes.keys()))  
>> León
```

```
print(max(ciudades_habitantes.values()))  
>> 1810645
```

```
print(max(lista_valores))  
>> 3125
```

* Nota: A partir de Python 3.7+, los diccionarios mantienen su orden de inserción.

Sets

Los **sets** son otro tipo de estructuras de datos. Se diferencian de listas, tuplas y diccionarios porque son una **colección mutable de elementos inmutables, no ordenados y sin datos repetidos**.

Mutable 

Ordenado 

Admite Duplicados 

métodos

add(item) agrega un elemento al set

```
mi_set_a.add(5)
print(mi_set_a)
>> {1, 2, "tres", 5}
```

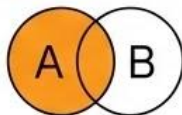
clear() remueve todos los elementos de un set

```
mi_set_a.clear()
print(mi_set_a)
>> set()
```

copy() retorna una copia del set

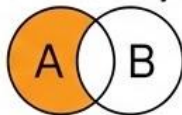
```
mi_set_c = mi_set_a.copy()
print(mi_set_c)
>> {1, 2, "tres"}
```

difference(set) retorna el set formado por todos los elementos que únicamente existen en el set A



```
mi_set_c = mi_set_a.difference(mi_set_b)
mi_set_c = mi_set_a.difference(mi_set_b)
print(mi_set_c)
>> {1, 2}
```

difference_update(set) remueve de A todos los elementos comunes a A y B

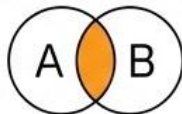


```
mi_set_a.difference_update(mi_set_b)
print(mi_set_a)
>> {1, 2}
```

discard(item) remueve un elemento del set

```
mi_set_a.discard("tres")
print(mi_set_a)
>> {1, 2}
```

intersection(set) retorna el set formado por todos los elementos que existen en A y B simultáneamente



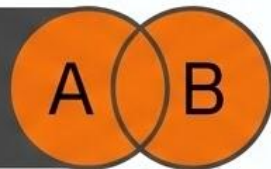
```
mi_set_c = mi_set_a.intersection(mi_set_b)
print(mi_set_c)
>> {'tres'}
```

Sets

```
mi_set_a = {1, 2, "tres"}    mi_set_b = {3, "tres"}
```

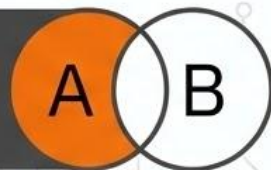
union(set) retorna un set resultado de combinar A y B (los datos duplicados se eliminan)

```
mi_set_c = mi_set_a.union(mi_set_b)
print(mi_set_c)
>> {1, 2, 3, 'tres'}
```



update(set) inserta en A los elementos de B

```
mi_set_a.update(mi_set_b)
print(mi_set_a)
>> {1, 2, 3, 'tres'}
```



Más sobre sets

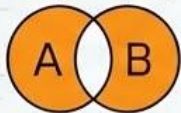
```
mi_set_a = {1, 2, "tres"} mi_set_b = {3, "tres"}
```

intersection_update(set) mantiene únicamente los elementos comunes a A y B



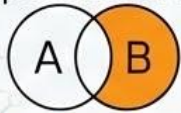
```
mi_set_b.intersection_update(mi_set_a)
print(mi_set_b)
>> {'tres'}
```

isdisjoint(set) devuelve True si A y B no tienen elementos en común



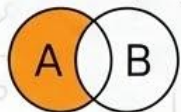
```
conjunto_disjunto = mi_set_a.isdisjoint(mi_set_b)
print(conjunto_disjunto)
>> False
```

issubset(set) devuelve True si todos los elementos de B están presentes en A



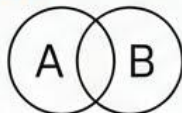
```
es_subset = mi_set_b.issubset(mi_set_a)
print(es_subset)
>> True
```

issuperset(set) devuelve True si A contiene todos los elementos de B



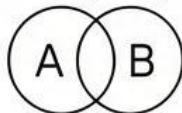
```
es_superset = mi_set_a.issuperset(mi_set_b)
print(es_superset)
>> True
```

pop() elimina y retorna un elemento al azar del set



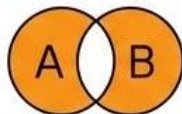
```
aleatorio = mi_set_a.pop()
print(aleatorio)
>> 1
```

remove(item) elimina un item del set, y arroja error si no existe



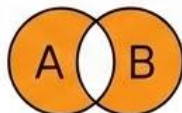
```
mi_set_a.remove("tres")
print(mi_set_a)
>> {2}
```

symmetric_difference(set) retorna todos los elementos de A y B, excepto aquellos que son comunes a los dos



```
mi_set_c = mi_set_b.symmetric_difference(mi_set_a)
print(mi_set_c)
>> {1, 2}
```

symmetric_difference_update(set) elimina los elementos comunes a A y B, agregando los que no están presentes en ambos a la vez



```
mi_set_b.symmetric_difference_update(mi_set_a)
print(mi_set_b)
>> {1, 2}
```


Enumerador

La función `enumerate()` nos facilita llevar la cuenta de las iteraciones, a través de un contador de índices de un iterable, que se puede utilizar de manera directa en un loop, o convertirse en una lista de tuplas con el método `list()`.

Cualquier objeto que
pueda ser iterado



`enumerate(iterable, inicio)`



Valor [int] de
inicio del índice
(por defecto
iniciado en 0)

```
print(list(enumerate("Hola")))
```

```
>> [(0, 'H'), (1, 'o'), (2, 'l'), (3, 'a')]
```

```
for indice, numero in enumerate([5.55, 6, 7.50]):  
    print(indice, numero)
```

```
>> 0 5.55
```

```
>> 1 6
```

```
>> 2 7.5
```

Ejercicio enumerador

El Problema

Imprime en pantalla frases como la siguiente:
"{nombre} se encuentra en el índice {índice}".
Donde nombre debe ser cada uno de los nombres de la lista a continuación, y el índice, obtenido mediante enumerate()

```
lista_nombres = ["Marcos", "Laura",  
"Mónica", "Javier", "Celina", "Marta",  
"Darío", "Emiliano", "Melisa"]
```

La Solución (Revelada)



Comprension de listas

La **comprehension** de listas ofrece una sintaxis más breve en la creación de una nueva lista basada en valores disponibles en otra secuencia. Vale la pena mencionar que la brevedad se logra a costo de una menor interpretabilidad.

cada elemento del iterable tuplas, sets, otras listas...

```
nueva_lista = [expresion for item in iterable if condicion == True]
```

fórmula matemática operación lógica

```
nueva_lista = [expresion if condicion == True else otra_expresion for item in iterable]
```

Ejemplo:

```
nueva_lista = [num**2 for num in range(10) if num < 5]
print(nueva_lista)
>> [0, 1, 4, 9, 16]
```


Ejercicio comprensión de listas

A partir de la lista valores, crea una lista `valores_pares` formada por los números de la lista valores que sean pares.

`valores = [1, 2, 3, 4, 5, 6, 9.5]`



Filtro: $n \% 2 == 0$



Ejercicio comprensión de diccionarios

Crea un diccionario usando comprensión de diccionarios y zip a partir de las listas dadas:

```
claves = ['nombre', 'edad', 'provincia']
```

```
valores = ['Jorge', 30, 'Misiones']
```



Solución:



Match - coincidencia de patrones

Sintaxis y Concepto General

Introducido en Python 3.10, `match` y `case` permiten la coincidencia de patrones estructurales, asociando acciones a acciones a formas específicas y tipos de datos complejos, más allá de simples valores literales.

```
match objeto:
    case <patron_1>:
        <accion_1>
    case <patron_2>:
        <accion_2>
    case <patron_3>:
        <accion_3>
    case _:
        <accion_comodin>
```

El carácter ``_`` es un comodín que actúa como coincidencia si ninguna de las anteriores se cumple.

Es posible detectar y deconstruir diferentes estructuras de datos, buscando coincidencias en su construcción.

Ejemplo de Estructuras Anidadas

```
cliente = {"nombre": "Ana Pérez",
           "edad": 30,
           "ocupacion": "Ingeniera"}

pelicula = {"titulo": "Inception",
            "fichaTecnica": {
                "protagonista": "Leonardo DiCaprio",
                "director": "Christopher Nolan"}
            }

frase = "Este es un mensaje sin formato"
elementos = [cliente, pelicula, frase, 100]

for i, elem in enumerate(elementos, start=1):
    match elem:
        case {"nombre": nombre, "edad": _, "ocupacion": ocupacion}:
            print(f"{i} - Cliente: {nombre} - Ocupacion: {ocupacion}")
        case {"titulo": titulo, "fichaTecnica": {"protagonista": _, "director": director}}:
            print(f"{i} - Película: {titulo} - Director: {director}")
        case str(texto):
            print(f"{i} - Palabra: {texto}")
        case _:
            print(f"{i} - No se que es esto!")
```


Ejercicio sobre patrones Match

Escribe un programa que recorra la lista e identifique cada elemento usando match

Entrada:

```
cliente = {"nombre": "Ana Pérez", "edad": 30, "ocupacion": "Ingeniera"}  
pelicula = {"titulo": "Inception", "fichaTecnica": {"protagonista": "Leonardo DiCaprio", "director": "Christopher Nolan"}}  
frase = "Este es un mensaje sin formato"  
  
elementos = [cliente, pelicula, frase, 100]
```

Solución:



Utilidades Python



Random

Generación de números y selecciones pseudo-aleatorias en Python

Configuración y Tipos Numéricos

```
from random import *  
# Importa todos los métodos
```

Proporciona funciones para generar valores o secuencias pseudo-aleatorias.

```
aleatorio = randint(1, 50)
```

Devuelve un **integer** entre dos valores (ambos límites incluidos).

```
aleatorio = round(uniform(1, 5), 1)
```

Devuelve un **float** entre un valor mínimo y uno máximo.

```
aleatorio = random()
```

Devuelve un **float** entre 0.0 y 1.0.

Operaciones con Secuencias

```
colores = ['azul', 'rojo', 'verde', 'amarillo']  
aleatorio = choice(colores)
```

Devuelve un **elemento al azar** de una secuencia (listas, tuplas, etc.).

```
numeros = list(range(5, 50, 5))  
shuffle(numeros)
```

Toma una secuencia mutable y **cambia el orden** de sus elementos aleatoriamente en el lugar.

Nota Importante

 La mecánica en cómo se generan dichos valores aleatorios viene en realidad predefinida en 'semillas'. Si bien sirve para todos los usos habituales, **no debe emplearse con fines de seguridad o criptográficos**, ya que son vulnerables.

Ejercicio sobre random

Enunciado del Problema

Utiliza el método `choice()` de la librería `random` para obtener un elemento al azar de la lista de nombres a continuación, y almacena el nombre escogido en una variable llamada `sorteo`.

```
nombres = ["Carlos", "Julia", "Nicole", "Laura", "Mailen"]
```



Manejo de Fechas en Python: Módulo `datetime`

El módulo estándar `datetime` provee clases para manipular fechas y horas. Las principales son `date` (solo fecha), `time` (solo hora) y `datetime` (ambas).

```
from datetime import datetime, date

# Fecha y hora actual
ahora = datetime.now()
print(f"Ahora: {ahora}")
# Salida: Ahora: 2023-10-27 10:30:45.123456

# Solo la fecha actual
hoy = date.today()
print(f"Hoy: {hoy}")
# Salida: Hoy: 2023-10-27
```



Formateo de Fechas: `strftime` y `strptime`

1. De Objeto a Cadena (`strftime`)

Convierte un objeto `datetime` a una cadena con un formato específico.

```
from datetime import datetime

ahora = datetime.now()
# Formato legible (Día/Mes/Año Hora:Minuto)
formato_legible = ahora.strftime("%d/%m/%Y %H:%M")
print(f"Formateado: {formato_legible}")
# Salida: Formateado: 27/10/2023 10:30
```

2. De Cadena a Objeto (`strptime`)

Analiza una cadena para crear un objeto `datetime`.

```
# Parsear desde cadena
fecha_str = "15-08-2024"
objeto_fecha = datetime.strptime(fecha_str, "%d-%m-%Y")
print(f"Objeto datetime: {objeto_fecha}")
# Salida: Objeto datetime: 2024-08-15 00:00:00
```



Aritmética de Fechas: Usando `timedelta`

La clase `timedelta` representa una duración o la diferencia entre dos fechas o tiempos. Permite realizar sumas y restas directas con objetos `datetime` o `date`.

```
from datetime import datetime, timedelta

ahora = datetime.now()
print(f"Hoy: {ahora.date()}")

# Calcular fecha futura (dentro de una semana)
una_semana = timedelta(weeks=1)
futuro = ahora + una_semana
print(f"En una semana: {futuro.date()}")

# Diferencia entre dos fechas
otra_fecha = datetime(2024, 1, 1)
diferencia = otra_fecha - ahora
print(f"Días hasta año nuevo: {diferencia.days}")
```



Más Ejemplos de Aritmética de Fechas

```
from datetime import datetime, date, timedelta
hoy = datetime.now()          # fecha y hora actuales
solo_fecha = date.today()     # solo la fecha actual
#Crear fechas especificas
fecha = date(2025, 5, 8)      # 8 de mayo de 2025
fecha_hora = datetime(2025, 5, 8, 14, 30) # con hora
#Extraer componentes de una fecha
print(fecha.day)             # Día
print(fecha.month)           # Mes
print(fecha.year)            # Año
# Formatear fechas
texto = fecha.strftime("%d/%m/%Y")    # '08/05/2023'
texto2 = fecha.strftime("%A, %B %d")   # 'Monday, May 08'
#Convertir texto en fecha
texto = "08/05/2025"
fecha = datetime.strptime(texto, "%d/%m/%Y")
```

```
#Sumar o restar días
maniana = date.today() + timedelta(days=1)
hace_una_semana = date.today() - timedelta(weeks=1)
#Diferencia entre fechas
inicio = date(2023, 5, 1)
fin = date(2023, 5, 8)
diferencia = fin - inicio          # timedelta
print(diferencia.days)              # 7
#Comparar fechas
if date.today() > fecha.date():
    print("La fecha ya pasó")
else:
    print("La fecha es en el futuro")
# Día de la semana
fecha.weekday()    # 0=Lunes, 6=Domingo
fecha.isoweekday() # 1=Lunes, 7=Domingo
```